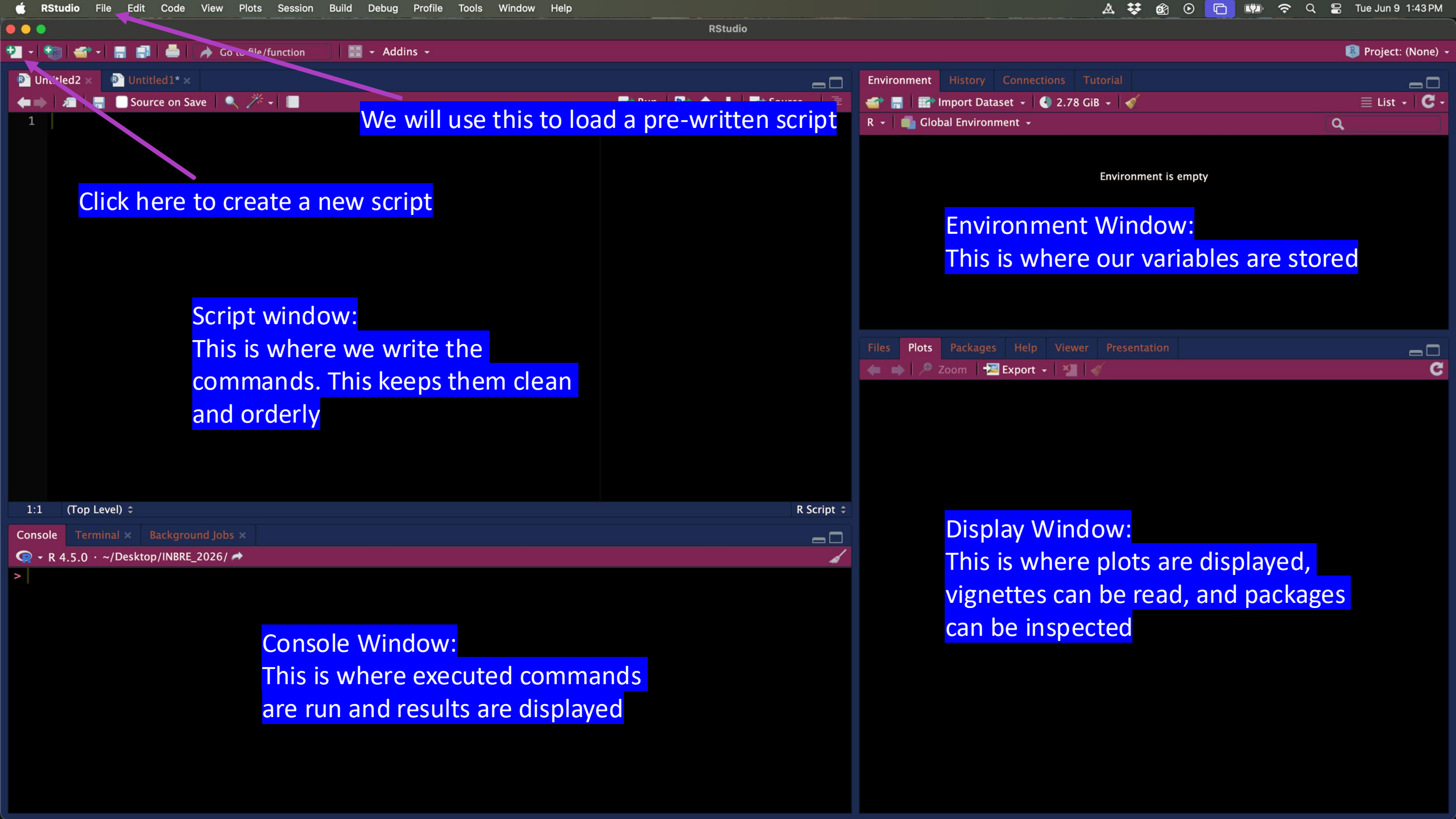


Today's outline

1. Cover Rstudio basics
2. Go through R script line by line
3. Run the script on your machine
4. Check results

R/RSTUDIO

- R is a “programming language”
- RStudio is an IDE (integrated development environment) for R
- More specifically, RStudio is a GUI (graphical user interface) that makes R easier to use
- R is a different language than BASH (what we use on the terminal and in bash scripts)
- Go ahead and open RStudio on your machine!



We will use this to load a pre-written script

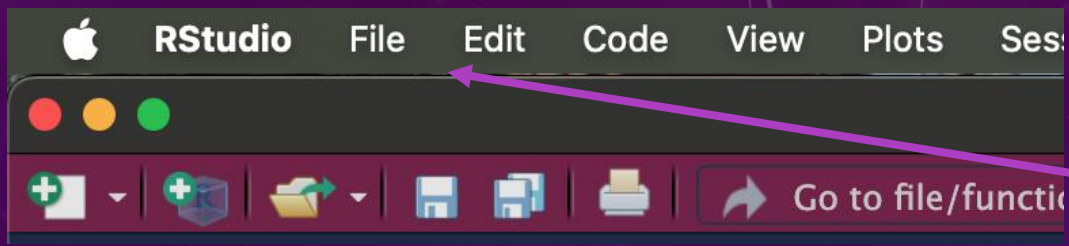
Click here to create a new script

Script window:
This is where we write the
commands. This keeps them clean
and orderly

Environment Window:
This is where our variables are stored

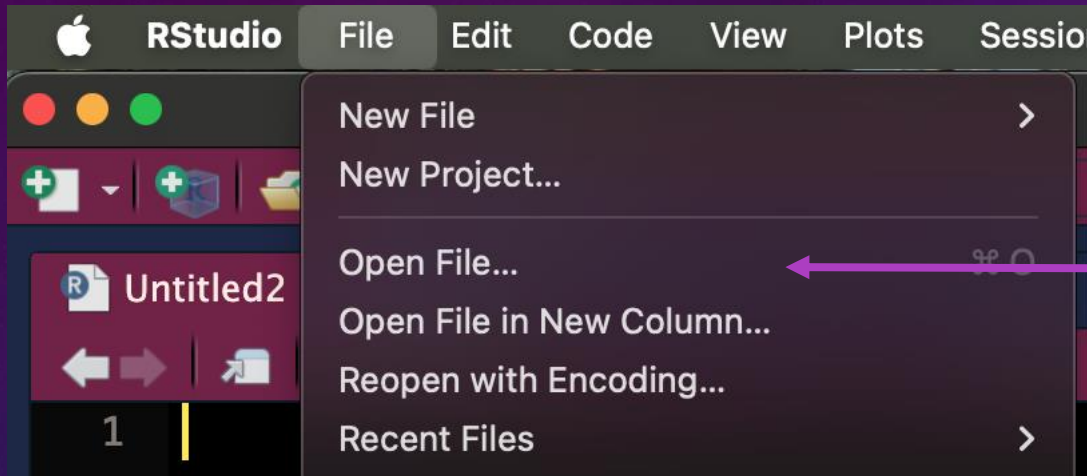
Display Window:
This is where plots are displayed,
vignettes can be read, and packages
can be inspected

Console Window:
This is where executed commands
are run and results are displayed

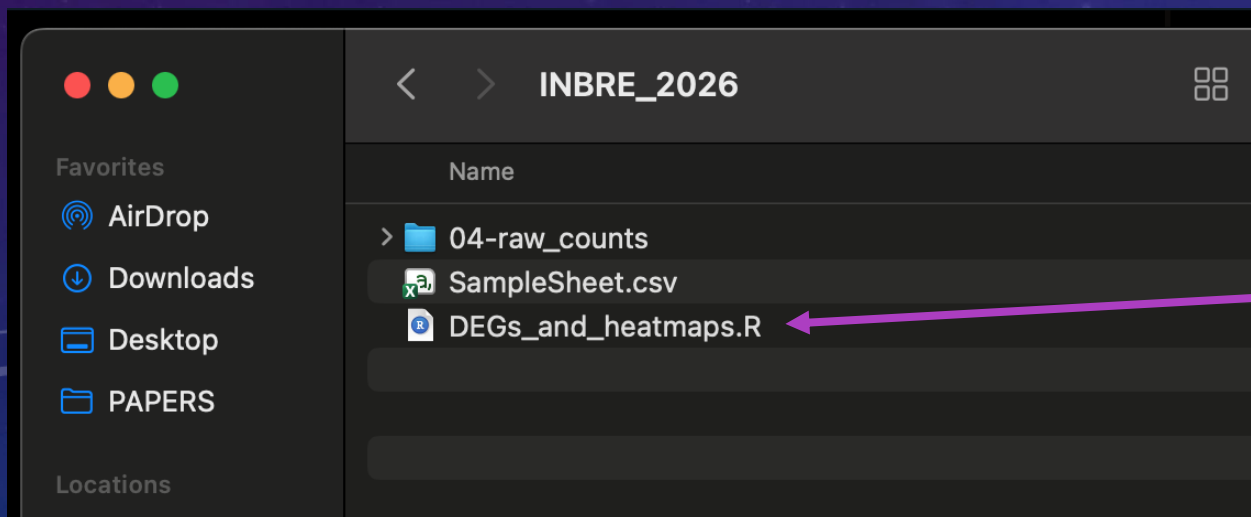


To open the pre-written script:

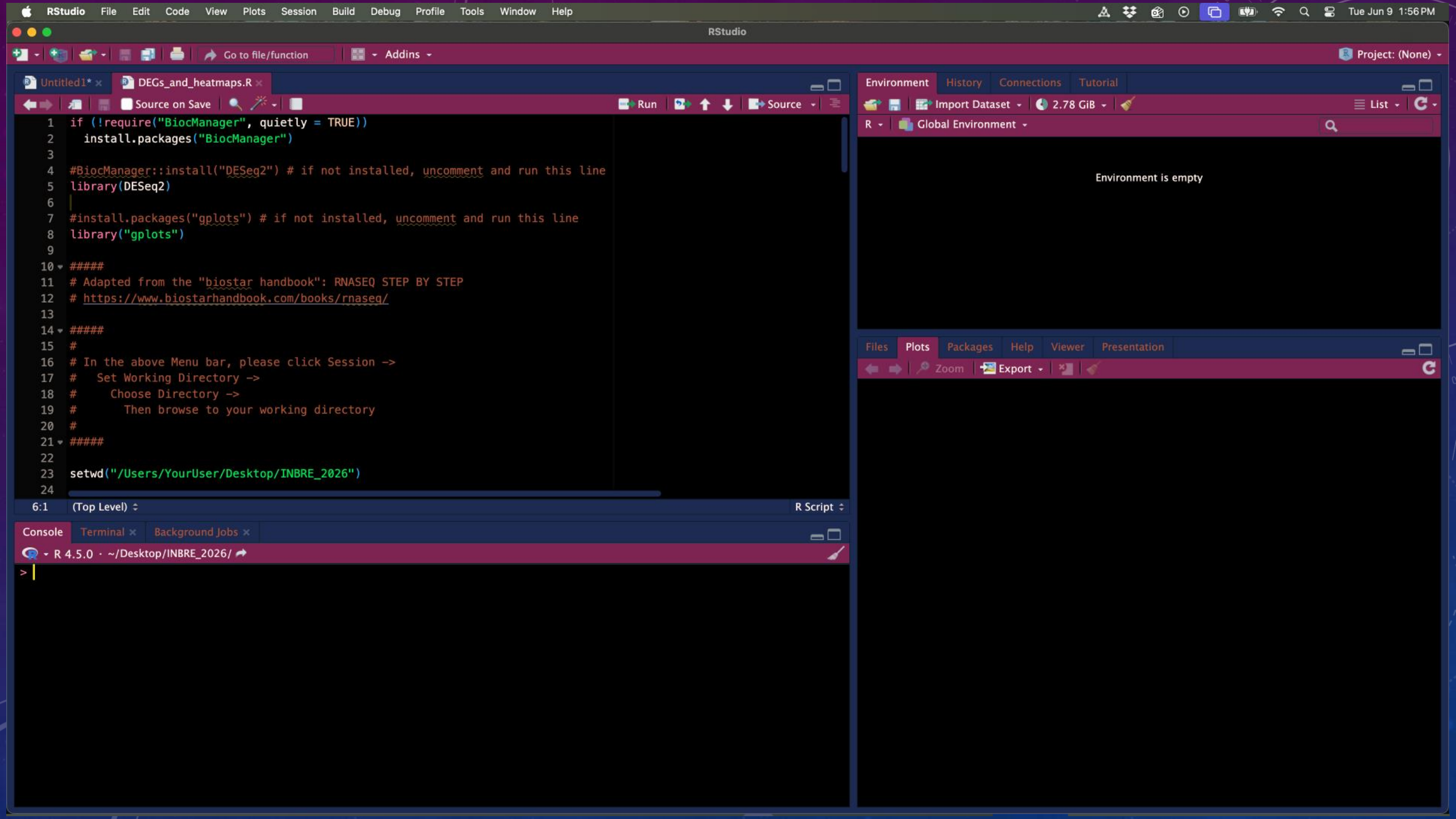
Click on File in the upper left corner



Click "Open File"



Navigate to the 'INBRE_2026' Folder.
Click on DEGs_and_heatmaps.R



HOW TO USE COOL STUFF IN R

- R comes with a ton of base tools that we can use
- This mostly includes reading/writing files, arithmetic, data manipulation, and basic statistics
- However! Similar to the cluster, many people have already written really cool code (packages) that we can use
- Today we will be using the R packages “DESeq2” to perform DE analysis, and “gplots” to make heatmaps

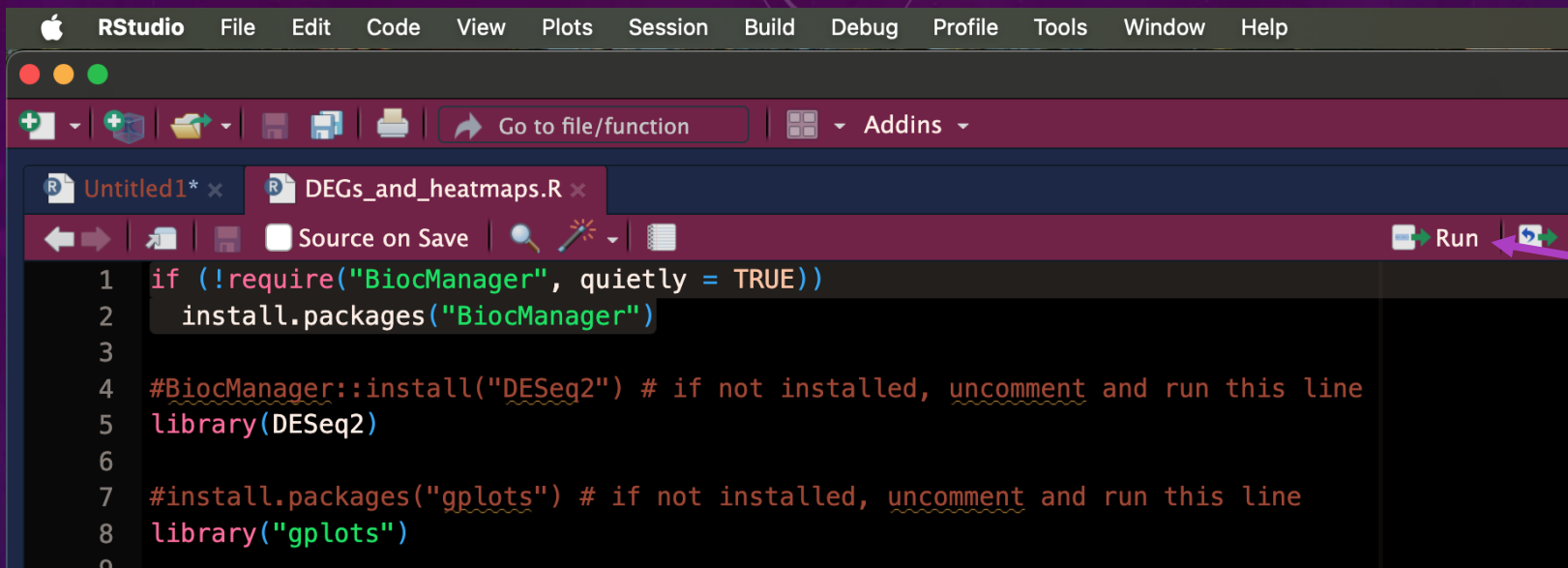
HOW TO USE COOL STUFF IN R, CONT.

- A “package” in R is similar to a module on the cluster: code someone else has already written and wrapped up in an easy-to-use set of R commands.
- To make use of these prewritten R commands, we must first install the package.
- We only have to install it once, but we have to load it every R session in which we want to use it.

HOW TO RUN COMMANDS IN RSTUDIO

1. If you click the “run” button, R will execute: A) what is highlighted, or B) the entire line your cursor is on.
2. If you press command+return (MAC) or control+enter (PC) R will execute the same as above.

This allows us to run code one line at a time. This is very helpful for writing and troubleshooting code.

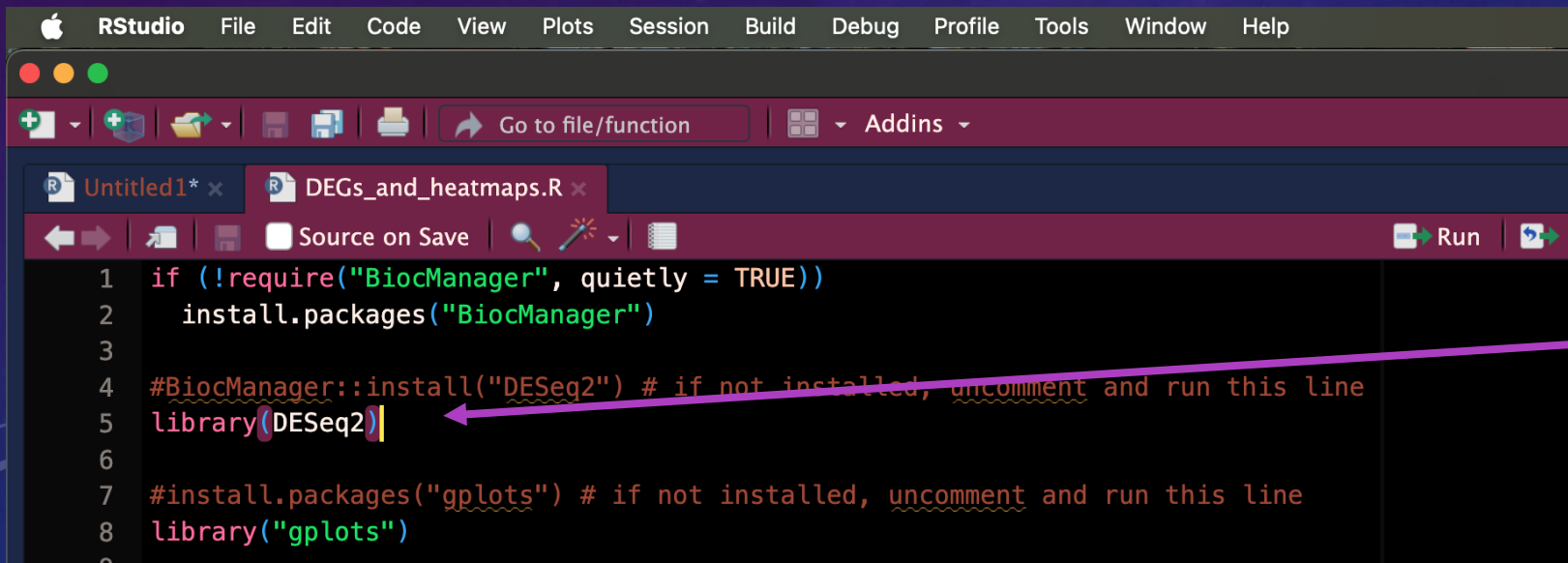


The image shows the RStudio interface with a menu bar (Apple, RStudio, File, Edit, Code, View, Plots, Session, Build, Debug, Profile, Tools, Window, Help) and a toolbar. The toolbar includes icons for file operations and a 'Run' button (a green play icon). Below the toolbar, two script tabs are open: 'Untitled1*' and 'DEGs_and_heatmaps.R'. The 'DEGs_and_heatmaps.R' tab is active, showing a script with the following code:

```
1 if (!require("BiocManager", quietly = TRUE))
2   install.packages("BiocManager")
3
4 #BiocManager::install("DESeq2") # if not installed, uncomment and run this line
5 library(DESeq2)
6
7 #install.packages("gplots") # if not installed, uncomment and run this line
8 library("gplots")
9
```

The code on lines 4 and 5 is highlighted in green. A pink arrow points from the 'Run' button in the toolbar to the highlighted code.

Clicking "Run" or pressing command+return (MAC) or control+enter (PC) will execute the highlighted section only.



The image shows the RStudio interface with the same menu bar and toolbar as the first image. The 'DEGs_and_heatmaps.R' tab is active, showing the same script. The code is:

```
1 if (!require("BiocManager", quietly = TRUE))
2   install.packages("BiocManager")
3
4 #BiocManager::install("DESeq2") # if not installed, uncomment and run this line
5 library(DESeq2)
6
7 #install.packages("gplots") # if not installed, uncomment and run this line
8 library("gplots")
9
```

The cursor is positioned at the end of line 5. A pink arrow points from the 'Run' button in the toolbar to the cursor.

If nothing is highlighted clicking "Run" will execute the entire line on which your cursor resides.

I TALK, YOU RUN CODE

- For this next section, I will show the R code on screen (with comments)
- YOU will run the lines of codes on your machine as we go through them.
- As always, ASK QUESTIONS!

```
1 if (!require("BiocManager", quietly = TRUE))
2   install.packages("BiocManager")
3
4 #BiocManager::install("DESeq2") # if not installed, uncomment and run this line
5 library(DESeq2)
6
7 #install.packages("gplots") # if not installed, uncomment and run this line
8 library("gplots")
9
10 #####
11 # Adapted from the "biostar handbook": RNASEQ STEP BY STEP
12 # https://www.biostarhandbook.com/books/rnaseq/
```

BiocManager is a suite of packages for bioinformatic analysis. DESeq2 is one of those packages.

If you have never installed "DESeq2" or "gplots" do it here.

Indicates a comment

You will need to run these every time this script is executed.

Untitled1*

DEGs_and_heatmaps.R

Source on Save

Run

↑

↓

Source

9

10 #####

11 # Adapted from the "biostar handbook": RNAseq STEP BY STEP

12 # <https://www.biostarhandbook.com/books/rnaseq/>

13

14 #####

15 #

16 # In the above Menu bar, please click Session ->

17 # Set Working Directory ->

18 # Choose Directory ->

19 # Then browse to your working directory

20 #

21 #####

22

23 setwd("/Users/YourUser/Desktop/INBRE_2026")

24

25 getwd() # Tells us where we are. Make sure it's where you want to be (This should be the location of your 'INBRE_20

26

27 # Must have counts data. Must be raw counts. Genes on the rows, samples on the columns.

28 rawCounts <- read.table("04-raw_counts/counts.txt", header = T) # this has too many extra columns

29

30 # Extract the gene names, so we don't lose them.

31 gene_names <- rawCounts\$Geneid

32

28:64

(Untitled)

R Script

Console

Terminal

Background Jobs

R 4.5.0 · ~/Desktop/INBRE_2026/

> setwd("/Users/dunnc/Desktop/INBRE_2026/")

> getwd()

[1] "/Users/dunnc/Desktop/INBRE_2026"

> rawCounts <- read.table("04-raw_counts/counts.txt", header = T)

> |

Environment

History

Connections

Tutorial

Import Dataset

2.79 GiB

List

Global Environment

Data

rawCounts

93 obs. of 12 variables

Files

Plots

Packages

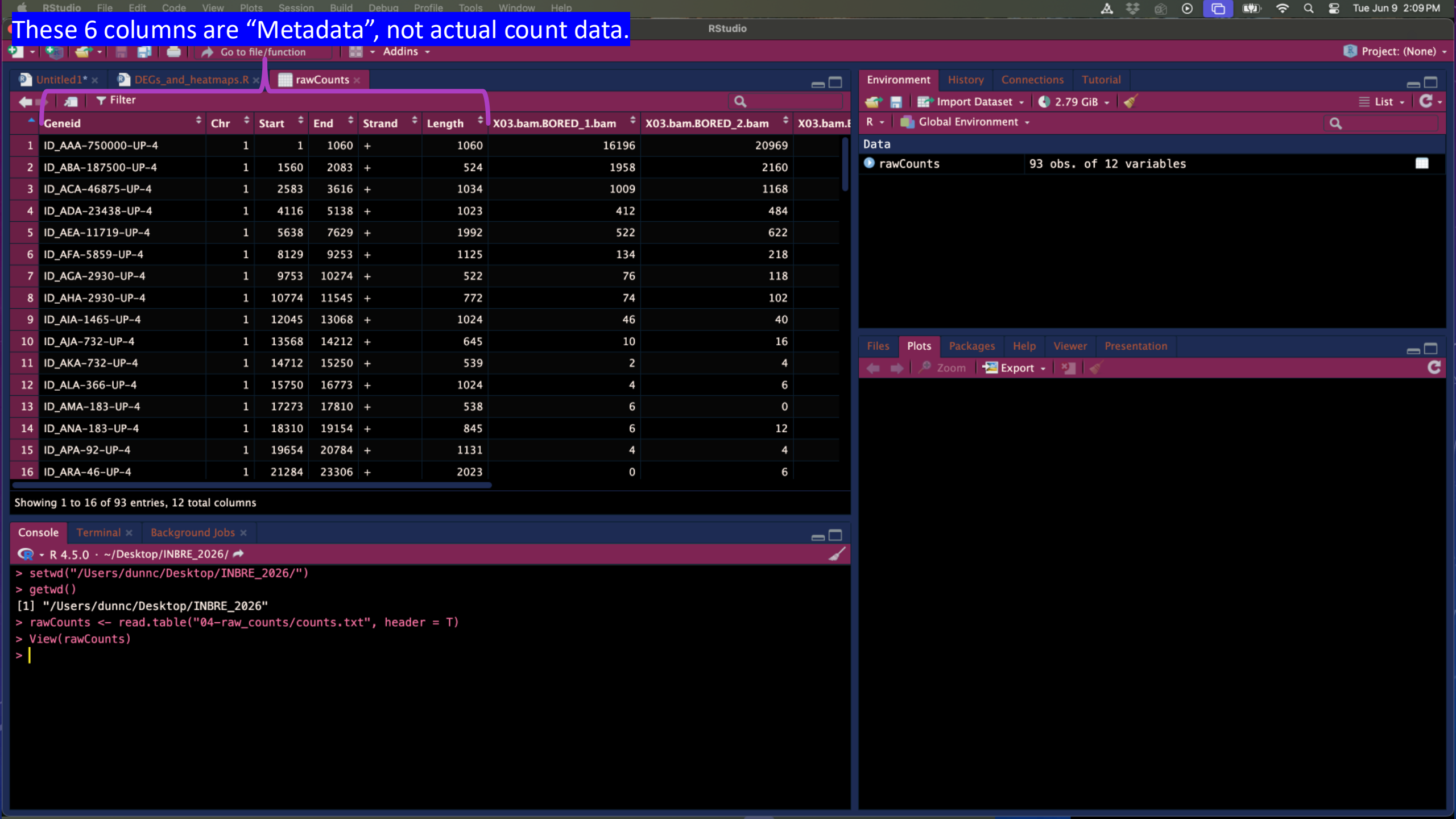
Help

Viewer

Presentation

Zoom

Export



RStudio File Edit Code View Plots Session Build Debug Profile Tools Window Help

Project: (None)

DEGs_and_heatmaps.R rawCounts

Source on Save Run Source

```
30 # Extract the gene names, so we don't lose them.
31 gene_names <- rawCounts$Geneid
32
33 # Remove the columns we don't need
34 rawCounts <- rawCounts[, -c(1:6)]
35
36 names(rawCounts) # I don't like how the sample names are formatted
37 temp <- names(rawCounts) # Save the column names as a variable
38
39 temp2 <- gsub("X03.bam.", "", temp) # Replace the 'X03.bam.' with nothing
40 temp3 <- gsub(".bam", "", temp2) # Replace the '.bam' with nothing
41
42
42:1 # (Untitled)
```

the "\$" will extract only the named column

the brackets are used to subset rows and columns

Environment History Connections Tutorial

Import Dataset 2.79 GiB

Global Environment

rawCounts 93 obs. of 6 variables

gene_names	chr [1:93] "ID_AAA-750000-UP-4" "ID_ABA-187500-UP-4" "ID_...
temp	chr [1:6] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X0...
temp2	chr [1:6] "BORED_1.bam" "BORED_2.bam" "BORED_3.bam" "EXCI...
temp3	chr [1:6] "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" "EXCI...

Files Plots Packages Help Viewer Presentation

Zoom Export

Console Terminal Background Jobs

R 4.5.0 ~ /Desktop/INBRE_2026/

```
> setwd("/Users/dunnc/Desktop/INBRE_2026/")
> getwd()
[1] "/Users/dunnc/Desktop/INBRE_2026"
> rawCounts <- read.table("04-raw_counts/counts.txt", header = T)
> View(rawCounts)
> # Extract the gene names, so we don't lose them.
> gene_names <- rawCounts$Geneid
> # Remove the columns we don't need
> rawCounts <- rawCounts[, -c(1:6)]
> names(rawCounts) # I don't like how the sample names are formatted
[1] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X03.bam.BORED_3.bam" "X03.bam.EXCITED_1.bam"
[5] "X03.bam.EXCITED_2.bam" "X03.bam.EXCITED_3.bam"
> temp <- names(rawCounts) # Save the column names as a variable
> temp2 <- gsub("X03.bam.", "", temp) # Replace the 'X03.bam.' with nothing
> temp3 <- gsub(".bam", "", temp2) # Replace the '.bam' with nothing
>
```

RStudio File Edit Code View Plots Session Build Debug Profile Tools Window Help

Go to file/function Addins

Project: (None)

Environment History Connections Tutorial

Import Dataset 2.79 GiB

R Global Environment

Data

rawCounts	93 obs. of 6 variables
sampleData	6 obs. of 2 variables

Values

gene_names	chr [1:93] "ID_AAA-750000-UP-4" "ID_ABA-187500-UP-4" "ID_...
temp	chr [1:6] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X0...
temp2	chr [1:6] "BORED_1.bam" "BORED_2.bam" "BORED_3.bam" "EXCI...
temp3	chr [1:6] "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" "EXCI...

Files Plots Packages Help Viewer Presentation

Zoom Export

```
40 temp3 <- gsub(".bam", "", temp2) # Replace the .bam with nothing
41
42 names(rawCounts) <- temp3 # Assign our nice clean names back to the columns of rawCounts
43 names(rawCounts) # Verify that we did what we wanted
44
45 row.names(rawCounts) <- gene_names # Assign our pretty gene names to columns
46
47 # Must have 'colData'. This is the classification for each subject (i.e. what groups we want to compare).
48 sampleData <- read.csv("SampleSheet.csv", header = T)
49
50 # Base level QC
51 rmn <- rowMeans(rawCounts)
52
```

51:10 (Untitled) R Script

Console Terminal Background Jobs

R 4.5.0 ~/Desktop/INBRE_2026

```
> setwd("~/Users/dunnc/Desktop/INBRE_2026/")
> getwd()
[1] "/Users/dunnc/Desktop/INBRE_2026"
> rawCounts <- read.table("04-raw_counts/counts.txt", header = T)
> View(rawCounts)
> # Extract the gene names, so we don't lose them.
> gene_names <- rawCounts$Geneid
> # Remove the columns we don't need
> rawCounts <- rawCounts[, -c(1:6)]
> names(rawCounts) # I don't like how the sample names are formatted
[1] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X03.bam.BORED_3.bam" "X03.bam.EXCITED_1.bam"
[5] "X03.bam.EXCITED_2.bam" "X03.bam.EXCITED_3.bam"
> temp <- names(rawCounts) # Save the column names as a variable
> temp2 <- gsub("X03.bam.", "", temp) # Replace the 'X03.bam.' with nothing
> temp3 <- gsub(".bam", "", temp2) # Replace the '.bam' with nothing
> names(rawCounts) <- temp3 # Assign our nice clean names back to the columns of rawCounts
> names(rawCounts) # Verify that we did what we wanted
[1] "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" "EXCITED_2" "EXCITED_3"
> row.names(rawCounts) <- gene_names
> # Must have 'colData'. This is the classification for each subject (i.e. what groups we want to compare).
> sampleData <- read.csv("SampleSheet.csv", header = T)
>
```

RStudioFileEditCodeViewPlotsSessionBuildDebugProfileToolsWindowHelp

Go to file/functionAddins

Project: (None)

Untitled1*DEGs_and_heatmaps.RsampleData

Filter

	Sample	Condition
1	BORED_1	Bored
2	BORED_2	Bored
3	BORED_3	Bored
4	EXCITED_1	Excited
5	EXCITED_2	Excited
6	EXCITED_3	Excited

Showing 1 to 6 of 6 entries, 2 total columns

EnvironmentHistoryConnectionsTutorial

Import Dataset2.79 GiB

List

RGlobal Environment

Data

rawCounts

93 obs. of 6 variables

sampleData

6 obs. of 2 variables

Values

gene_names

chr [1:93] "ID_AAA-750000-UP-4" "ID_ABA-187500-UP-4" "ID_...

temp

chr [1:6] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X0...

temp2

chr [1:6] "BORED_1.bam" "BORED_2.bam" "BORED_3.bam" "EXCI...

temp3

chr [1:6] "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" "EXCI...

ConsoleTerminalBackground Jobs

R 4.5.0 · ~/Desktop/INBRE_2026/

> View(sampleData)

>

FilesPlotsPackagesHelpViewerPresentation

ZoomExport

FileEditCodeViewPlotsSessionBuildDebugProfileToolsWindowHelp

RStudio

Go to file/functionAddins

Project: (None)

Untitled1*DEGs_and_heatmaps.RsampleData

RunSource

```
49
50 # Base level QC
51 rmn <- rowMeans(rawCounts)
52
53 rawCounts2 <- rawCounts[rmn >= 0.5,] # I would usually be more strict
54
55 # Look at how our data is stored
56 str(rawCounts2)
57 str(sampleData)
58
59 # Create the DEseq2DataSet object
60 # Load data into the DEseq2 object
61 deseq2Data <- DESeqDataSetFromMatrix(countData = rawCounts2, colData = sampleData, design = ~ Condition) ## We can
62
63
59:1 (Untitled) R Script
```

ConsoleTerminalBackground Jobs

R 4.5.0 ~ /Desktop/INBRE_2026/

> View(sampleData)
> # Base level QC
> rmn <- rowMeans(rawCounts)
> rawCounts2 <- rawCounts[rmn >= 0.5,] # I would usually be more strict
> # Look at how our data is stored
> str(rawCounts2)
'data.frame': 78 obs. of 6 variables:
 \$ BORED_1 : int 16196 1958 1009 412 522 134 76 74 46 10 ...
 \$ BORED_2 : int 20969 2160 1168 484 622 218 118 102 40 16 ...
 \$ BORED_3 : int 17650 2018 1010 480 564 196 72 64 26 8 ...
 \$ EXCITED_1: int 139542 18398 8912 3840 4226 1436 626 630 398 66 ...
 \$ EXCITED_2: int 89822 13566 6128 2456 2824 932 410 382 222 42 ...
 \$ EXCITED_3: int 114177 14715 7338 3338 3934 1252 544 476 300 42 ...
> str(sampleData)
'data.frame': 6 obs. of 2 variables:
 \$ Sample : chr "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" ...
 \$ Condition: chr "Bored" "Bored" "Bored" "Excited" ...
> |

EnvironmentHistoryConnectionsTutorial

Import Dataset2.79 GiB

List

RGlobal Environment

Data

rawCounts	93 obs. of 6 variables
rawCounts2	78 obs. of 6 variables
	6 obs. of 2 variables

gene_nameschr [1:93] "ID_AAA-750000-UP-4" "ID_ABA-187500-UP-4" "ID_...

rmnNamed num [1:93] 66393 8802 4261 1835 2115 ...

tempchr [1:6] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X0...

temp2chr [1:6] "BORED_1.bam" "BORED_2.bam" "BORED_3.bam" "EXCI...

temp3chr [1:6] "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" "EXCI...

FilesPlotsPackagesHelpViewerPresentation

ZoomExport

"str" is a command to show you the structure of the data.

For DESeq2, the count data must be integers.

56 str(rawCounts2)
57 str(sampleData)
58
59 # Create the DEseq2DataSet object
60 # Load data into the DEseq2 object
61 deseq2Data <- DESeqDataSetFromMatrix(countData = rawCounts2, colData = sampleData, design = ~ Condition) ## We can
62
63 # Do 'DE analysis'
64 deseq2Data <- DESeq(deseq2Data) # This step is the bottleneck.
65
66 # Make the PC plot to check for groupings
67 # Transform the data to meet statistical assumptions
68 vst <- varianceStabilizingTransformation(deseq2Data, blind = F)
69
70
66:1 # (Untitled) ↕

Console

Terminal × Background Jobs ×

R 4.5.0 · ~/Desktop/INBRE_2026/ ↕

\$ BORED_3 : int 17650 2018 1010 480 564 196 72 64 26 8 ...
\$ EXCITED_1: int 139542 18398 8912 3840 4226 1436 626 630 398 66 ...
\$ EXCITED_2: int 89822 13566 6128 2456 2824 932 410 382 222 42 ...
\$ EXCITED_3: int 114177 14715 7338 3338 3934 1252 544 476 300 42 ...
> str(sampleData)
'data.frame': 6 obs. of 2 variables:
 \$ Sample : chr "BORED_1" "BORED_2" "BORED_3" "EXCITED_1" ...
 \$ Condition: chr "Bored" "Bored" "Bored" "Excited" ...
> # Create the DEseq2DataSet object
> # Load data into the DEseq2 object
> deseq2Data <- DESeqDataSetFromMatrix(countData = rawCounts2, colData = sampleData, design = ~ Condition) ## We can include covariates here as well
Warning message:
In DESeqDataSet(se, design = design, ignoreRank) :
 some variables in design formula are characters, converting to factors
> # Do 'DE analysis'
> deseq2Data <- DESeq(deseq2Data) # This step is the bottleneck.
estimating size factors
estimating dispersions
gene-wise dispersion estimates
mean-dispersion relationship
final dispersion estimates
fitting model and testing
> |

Environment

History

Connections

Tutorial

Import Dataset ↕ 2.79 GiB ↕

List ↕ ↻

R ↕ Global Environment ↕

Search

Data

deseq2Data Formal class DESeqDataSet ↕

rawCounts 93 obs. of 6 variables ↕

rawCounts2 78 obs. of 6 variables ↕

sampleData 6 obs. of 2 variables ↕

Values

gene_names chr [1:93] "ID_AAA-750000-UP-4" "ID_ABA-187500-UP-4" "ID_...

rmn Named num [1:93] 66393 8802 4261 1835 2115 ...

temp chr [1:6] "X03.bam.BORED_1.bam" "X03.bam.BORED_2.bam" "X0...

temp2 chr [1:6] "BORED_1.bam" "BORED_2.bam" "BORED_3.bam" "EXCI...

Files

Plots

Packages

Help

Viewer

Presentation

Zoom ↕

Export ↕ ↻

These 2 lines are the “heavy lifters”. They do all of the math and organization.

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Window Help

Go to file/function Addins

Project: (None)

Environment History Connections Tutorial

Import Dataset 2.79 GiB

Global Environment

deseq2counts num [1:78, 1:6] 21902 2648 1364 557 706 ...

deseq2Data Formal class DESeqDataSet

rawCounts 93 obs. of 6 variables

rawCounts2 78 obs. of 6 variables

sampleData 6 obs. of 2 variables

rmn Named num [1:93] 66393 8802 4261 1835 2115 ...

```
68 vst <- varianceStabilizingTransformation(deseq2Data, blind = F)
69
70 # Plot PC1 vs PC2
71 pdf("PCA.pdf")
72 plotPCA(vst, intgroup = "Condition")
73 dev.off()
74
75 # Save the normalized counts to use later
76 deseq2counts <- counts(deseq2Data, normalized = T)
77
78 #####
79 ## Here we make a specific comparison: Bored vs Excited
80 #####
81 #
```

78:1 (Untitled) R Script

Console Terminal Background Jobs

R 4.5.0 ~/Desktop/INBRE_2026/

Warning message:

In DESeqDataSet(se, design = design, ignoreRank) :

some variables in design formula are characters, converting to factors

```
> # Do 'DE analysis'
> deseq2Data <- DESeq(deseq2Data) # This step is the bottleneck.
estimating size factors
estimating dispersions
gene-wise dispersion estimates
mean-dispersion relationship
final dispersion estimates
fitting model and testing
> # Make the PC plot to check for groupings
> # Transform the data to meet statistical assumptions
> vst <- varianceStabilizingTransformation(deseq2Data, blind = F)
> # Plot PC1 vs PC2
> pdf("PCA.pdf")
> plotPCA(vst, intgroup = "Condition")
using ntop=500 top features by variance
> dev.off()
null device
1
> # Save the normalized counts to use later
> deseq2counts <- counts(deseq2Data, normalized = T)
>
```

This is a normalization designed for PCA

This will produce the actual plot

Here we can store the DESeq2 normalized counts for reference or downstream stuff

The "results" command is how we extract specific comparisons

We convert to a data frame for ease of viewing

```
//
78 > #####
79 ## Here we make a specific comparison
80 > #####
81 #                               design variable, experimental, control/reference
82 Results <- results(deseq2Data, cooksCutoff = F, contrast = c("Condition", "Excited", "Bored"))
83 ResultsDF <- as.data.frame(Results)
84
85 # Let's add some useful info
86
87 # Calculate the mean count for each gene in each group
88 bored_avg <- apply(deseq2counts[, c(1,2,3)], 1, mean)
89 excited_avg <- apply(deseq2counts[, c(4,5,6)], 1, mean)
90
91
79:2 ## (Untitled) ↕
```

```
R 4.5.0 · ~/Desktop/INBRE_2026/ ↗
estimating dispersions
gene-wise dispersion estimates
mean-dispersion relationship
final dispersion estimates
fitting model and testing
> # Make the PC plot to check for groupings
> # Transform the data to meet statistical assumptions
> vst <- varianceStabilizingTransformation(deseq2Data, blind = F)
> # Plot PC1 vs PC2
> pdf("PCA.pdf")
> plotPCA(vst, intgroup = "Condition")
using ntop=500 top features by variance
> dev.off()
null device
1
> # Save the normalized counts to use later
> deseq2counts <- counts(deseq2Data, normalized = T)
> #####
> ## Here we make a specific comparison: Bored vs Excited
> #####
> #                               design variable, experimental, control/reference
> Results <- results(deseq2Data, cooksCutoff = F, contrast = c("Condition", "Excited", "Bored"))
> ResultsDF <- as.data.frame(Results)
> |
```

Environment			History	Connections	Tutorial
R			Import Dataset	2.79 GiB	
Global Environment					
Data					
deseq2counts	num [1:78, 1:6]	21902 2648 1364 557 706 ...			
deseq2Data	Formal class DESeqDataSet				
rawCounts	93 obs. of 6 variables				
rawCounts2	78 obs. of 6 variables				
Results	Formal class DESeqResults				
ResultsDF	78 obs. of 6 variables				
sampleData	6 obs. of 2 variables				
vst	Formal class DESeqTransform				
Values					
Files			Plots	Packages	Help
Zoom			Export	Viewer	Presentation

Showing 1 to 10 of 78 entries, 6 total columns

gene-wise dispersion estimates









Data

Navigation icons: back, forward, search, zoom, export, print, and refresh.

```
> plotPCA(vst, intgroup = "Condition")
```


RStudio

FileEditCodeViewPlotsSessionBuildDebugProfileToolsWindowHelp

Go to file/functionAddins

Project: (None)

Untitled1*DEGs_and_heatmaps.RResultsDF

Filter

	baseMean	log2FoldChange	lfcSE	stat	pvalue	padj	BoredMean	ExcitedMean	FC
JP-4	5.770827e+04	2.16772635	0.07970463	27.19699341	7.049097e-163	4.229458e-161	21011.460	94405.090	4.4931473
JP-4	7.676007e+03	2.45639191	0.10261835	23.93716103	1.257243e-126	1.885865e-125	2366.942	12985.072	5.484239
JP-4	3.705993e+03	2.33322387	0.09709311	24.03078839	1.325858e-127	2.651717e-126	1228.215	6183.771	5.0393018
JP-4	1.585968e+03	2.32481303	0.09143806	25.42500498	1.334529e-142	4.003588e-141	528.338	2643.598	5.0100084
JP-4	1.839169e+03	2.20415889	0.09294687	23.71418172	2.574639e-124	3.089567e-123	656.573	3021.765	4.6080581
JP-4	6.009779e+02	2.25468052	0.12545475	17.97206191	3.225022e-72	3.225022e-71	207.449	994.507	4.7722860
JP-4	2.679961e+02	2.09527982	0.17740567	11.81067026	3.438095e-32	2.946938e-31	101.573	434.419	4.2730903
JP-4	2.497368e+02	2.14655848	0.18406674	11.66184888	1.996592e-31	1.497444e-30	92.179	407.295	4.4277031
JP-4	1.471078e+02	2.52569732	0.24472693	10.32047140	5.694289e-25	3.416574e-24	44.128	250.088	5.7585170

Showing 1 to 9 of 78 entries, 9 total columns

ConsoleTerminalBackground Jobs

R 4.5.0 · ~/Desktop/INBRE_2026/

using ntop=500 top features by variance
> dev.off()
null device
1
> # Save the normalized counts to use later
> deseq2counts <- counts(deseq2Data, normalized = T)
> #####
> ## Here we make a specific comparison: Bored vs Excited
> #####
> #
design variable, experimental, control/reference
> Results <- results(deseq2Data, cooksCutoff = F, contrast = c("Condition", "Excited", "Bored"))
> ResultsDF <- as.data.frame(Results)
> View(ResultsDF)
> # Calculate the mean count for each gene in each group
> bored_avg <- apply(deseq2counts[, c(1,2,3)], 1, mean)
> excited_avg <- apply(deseq2counts[, c(4,5,6)], 1, mean)
> # Calculate FC in addition to the log2FC
> FC <- 2^ResultsDF\$log2FoldChange
> # Attach new values to the 'Results' data frame
> ResultsDF\$BoredMean <- round(bored_avg, 3)
> ResultsDF\$ExcitedMean <- round(excited_avg, 3)
> ResultsDF\$FC <- FC
> View(ResultsDF)
>

EnvironmentHistoryConnectionsTutorial

Import Dataset2.79 GiB

List

RGlobal Environment

rawCounts93 obs. of 6 variables
rawCounts278 obs. of 6 variables
ResultsFormal class DESeqResults
ResultsDF78 obs. of 9 variables
sampleData6 obs. of 2 variables
vstFormal class DESeqTransform

Values

bored_avgNamed num [1:78] 21011 2367 1228 528 657 ...
excited_avgNamed num [1:78] 94405 12985 6184 2644 3022 ...
FCnum [1:78] 4.49 5.49 5.04 5.01 4.61 ...

FilesPlotsPackagesHelpViewerPresentation

ZoomExport

Always check that your FC means what you think it does!!!

RStudio interface showing R code for differential gene expression analysis. The code filters for significantly differentially expressed (DE) genes based on log2 fold change and adjusted p-value.

Code Snippets:

```
97 ResultsDF$FC <- FC
98
99 # Filter to only 'differentially expressed' genes
100 # Defined here as log2FC > 0.75 or < -0.75, and an adjusted p.value < 0.05
101 Results_Filt <- ResultsDF[which(abs(ResultsDF$log2FoldChange) > 0.75 & ResultsDF$padj < 0.05),]
102
103 # Write both the full and filtered tables to file
104 write.csv(ResultsDF, "All_genes_results.csv", quote = F, row.names = T)
105 write.csv(Results_Filt, "DE_genes_results.csv", quote = F, row.names = T)
106
107 #####
108 # Now we proceed to heatmaps
109 #####
110
111
```

Environment Panel:

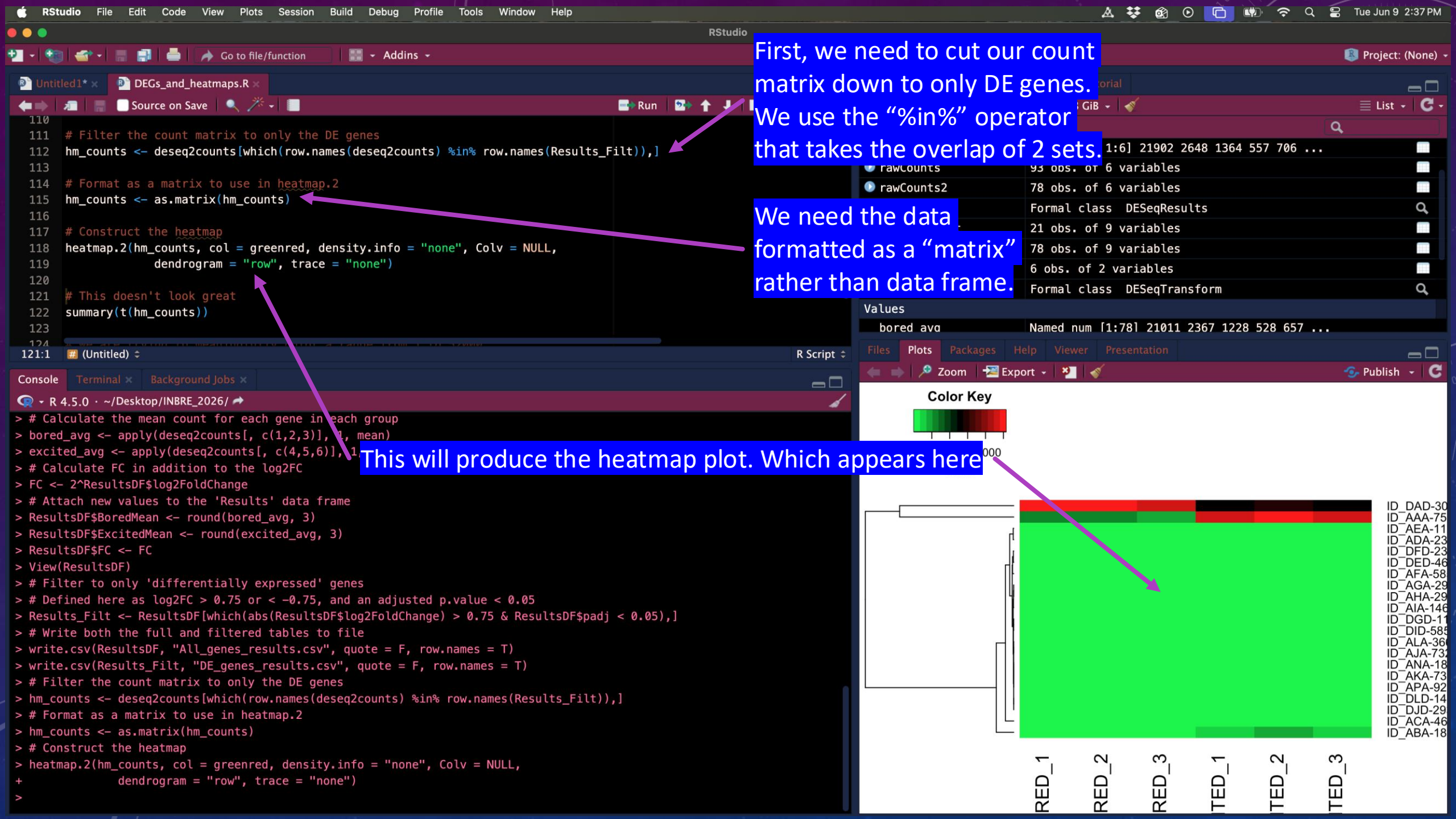
Variable	Class	Value
ResultsDF	data.frame	78 obs. of 9 variables
sampleData	data.frame	6 obs. of 2 variables
vst	Formal class DESeqTransform	

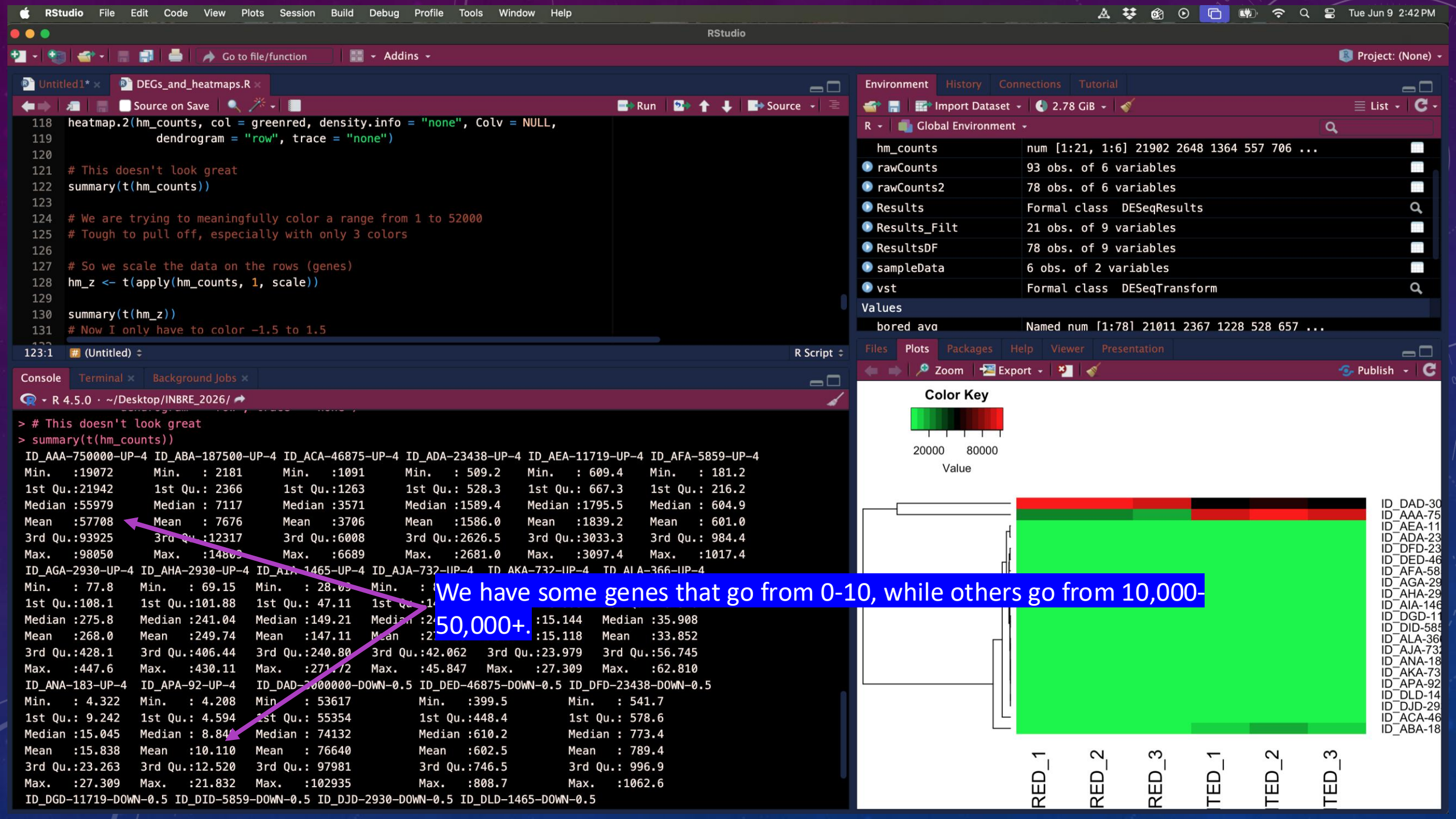
Console Output:

```
> #####
> ## Here we make a specific comparison: Bored vs Excited
> #####
> #
> # design variable, experimental, control/reference
> Results <- results(deseq2Data, cooksCutoff = F, contrast = c("Condition", "Excited", "Bored"))
> ResultsDF <- as.data.frame(Results)
> View(ResultsDF)
> # Calculate the mean count for each gene in each group
> bored_avg <- apply(deseq2counts[, c(1,2,3)], 1, mean)
> excited_avg <- apply(deseq2counts[, c(4,5,6)], 1, mean)
> # Calculate FC in addition to the log2FC
> FC <- 2^ResultsDF$log2FoldChange
> # Attach new values to the 'Results' data frame
> ResultsDF$BoredMean <- round(bored_avg, 3)
> ResultsDF$ExcitedMean <- round(excited_avg, 3)
> ResultsDF$FC <- FC
> View(ResultsDF)
> # Filter to only 'differentially expressed' genes
> # Defined here as log2FC > 0.75 or < -0.75, and an adjusted p.value < 0.05
> Results_Filt <- ResultsDF[which(abs(ResultsDF$log2FoldChange) > 0.75 & ResultsDF$padj < 0.05),]
> # Write both the full and filtered tables to file
> write.csv(ResultsDF, "All_genes_results.csv", quote = F, row.names = T)
> write.csv(Results_Filt, "DE_genes_results.csv", quote = F, row.names = T)
>
```

Annotations:

- We can filter down to only "significantly" DE genes. We usually define "significance" by 2 criteria: a statistical significance and practical significance.
- Then we write our results to file!!



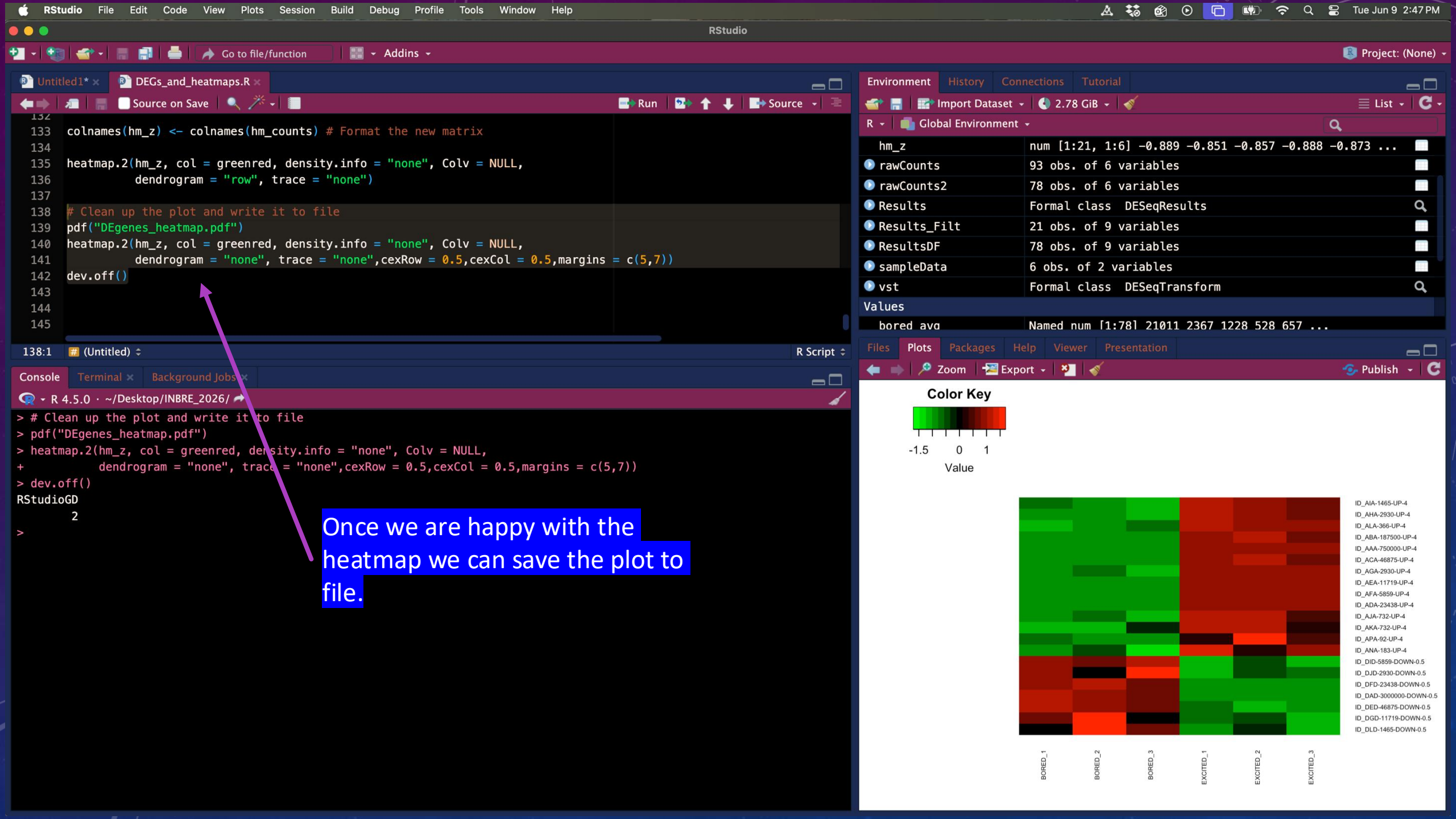



```
123
124 # We are trying to meaningfully color a range from 1 to 52000
125 # Tough to pull off, especially with only 3 colors
126
127 # So we scale the data on the rows (genes)
128 hm_z <- t(apply(hm_counts, 1, scale))
129
130 summary(t(hm_z))
131 # Now I only have to color -1.5 to 1.5
132
133 colnames(hm_z) <- colnames(hm_counts) # Format the new matrix
134
135 heatmap.2(hm_z, col = greenred, density.info = "none", Colv = NULL,
136           dendrogram = "row", trace = "none")
137
131:39 (Untitled) R Script
```

```
Min. : -1.179299 Min. : -1.03131 Min. : -1.2500 Min. : -0.8540 Min. : -0.9638
1st Qu.: -0.828444 1st Qu.: -0.91866 1st Qu.: -0.7160 1st Qu.: -0.7982 1st Qu.: -0.8911
Median : 0.002474 Median : 0.07456 Median : -0.0861 Median : -0.1824 Median : -0.1050
Mean : 0.000000 Mean : 0.00000 Mean : 0.0000 Mean : 0.0000 Mean : 0.0000
3rd Qu.: 0.841836 3rd Qu.: 0.83008 3rd Qu.: 0.8059 3rd Qu.: 0.3489 3rd Qu.: 0.8934
Max. : 1.158144 Max. : 1.05000 Max. : 1.2450 Max. : 1.6963 Max. : 1.1008
ID_DED-46875-DOWN-0.5 ID_DFD-23438-DOWN-0.5 ID_DGD-11719-DOWN-0.5 ID_DID-5859-DOWN-0.5 ID_DJD-2930-DOWN-0.5
Min. : -1.12704 Min. : -1.01623 Min. : -1.1063 Min. : -1.06461 Min. : -1.2043
1st Qu.: -0.85591 1st Qu.: -0.86492 1st Qu.: -0.6677 1st Qu.: -0.91223 1st Qu.: -0.5709
Median : 0.04284 Median : -0.06535 Median : -0.2289 Median : 0.02951 Median : -0.2746
Mean : 0.00000 Mean : 0.00000 Mean : 0.0000 Mean : 0.00000 Mean : 0.0000
3rd Qu.: 0.79959 3rd Qu.: 0.85164 3rd Qu.: 0.5455 3rd Qu.: 0.81358 3rd Qu.: 0.6398
Max. : 1.14502 Max. : 1.12107 Max. : 1.5743 Max. : 1.15680 Max. : 1.4786
ID_DLD-1465-DOWN-0.5
Min. : -1.06846
1st Qu.: -0.77262
Median : -0.06153
Mean : 0.00000
3rd Qu.: 0.44160
Max. : 1.59185
> colnames(hm_z) <- colnames(hm_counts) # Format the new matrix
> heatmap.2(hm_z, col = greenred, density.info = "none", Colv = NULL,
+           dendrogram = "row", trace = "none")
>
```

If we scale the data (by genes, not by subject), it will put all gene expression values in a similar range.





CONGRATULATIONS! YOU JUST DID A DE ANALYSIS!

- Last time: QC'd FASTQs, aligned FASTQs, converted to .bam, sorted, indexed, and calculated a count matrix.
- This time: Loaded count matrix and sampleData into R, QC'd counts, performed DE, filtered results, added useful columns, visualized with heatmaps.